

Material de estudo — Event loop e libuv

Módulo: Node.js Profundo · **Tópico:** Event loop e libuv **Âncora:** seu runtime real em produção — onde o event loop ajudou ou atrapalhou. **Como ler:** cada seção abre com uma pergunta. Tente respondê-la de cabeça antes de seguir — a explicação vem logo depois, com a profundidade que a entrevista vai cobrar. O objetivo é construir o modelo por raciocínio, não decorar.

1. Como um servidor de uma thread só atende milhares de conexões ao mesmo tempo?

Pare e pense antes de ler. Se o JavaScript roda numa thread única, e um servidor precisa atender 5 mil clientes simultâneos, como isso não é impossível?

A pegadinha está na palavra "atender". Na maioria dos backends, atender uma requisição é **quase toda espera**: espera o banco, espera o cache, espera outro serviço. O trabalho de CPU real — montar a resposta — leva microssegundos; a espera leva milissegundos. Se a thread pudesse *largar* uma requisição enquanto ela espera e ir cuidar de outra, uma thread só daria conta de milhares.

É exatamente isso que o event loop faz. Ele não executa mil coisas ao mesmo tempo — ele nunca para de trocar de tarefa enquanto as outras esperam. A ilusão de concorrência vem de a espera ser delegada para fora do JavaScript.

Aprofundando (o que a entrevista cobra): compare com o modelo alternativo, uma thread por conexão (Apache clássico). 10 mil conexões custam 10 mil threads, cada uma com sua pilha de memória e overhead de context-switch — o custo cresce linear com os clientes. O modelo de event loop atende as mesmas 10 mil conexões com uma thread de JS, porque o tempo dominante numa aplicação típica não é CPU, é espera por I/O. Enquanto uma requisição espera o banco responder, a thread está livre para processar outras cem.

Quem cuida da espera? Uma biblioteca em C chamada `libuv`. Ela fala com o sistema operacional (epoll no Linux, kqueue no macOS, IOCP no Windows) para ser avisada quando cada I/O termina, e mantém o loop que decide o que rodar em seguida. Quando dizem "o event loop do Node", é o loop do libuv com callbacks de JavaScript pendurados nele.

2. Em que ordem, exatamente, o Node executa as coisas?

Tente prever a saída deste código antes de continuar:

```
console.log('A')
setTimeout(() => console.log('B'), 0)
Promise.resolve().then(() => console.log('C'))
process.nextTick(() => console.log('D'))
console.log('E')
```

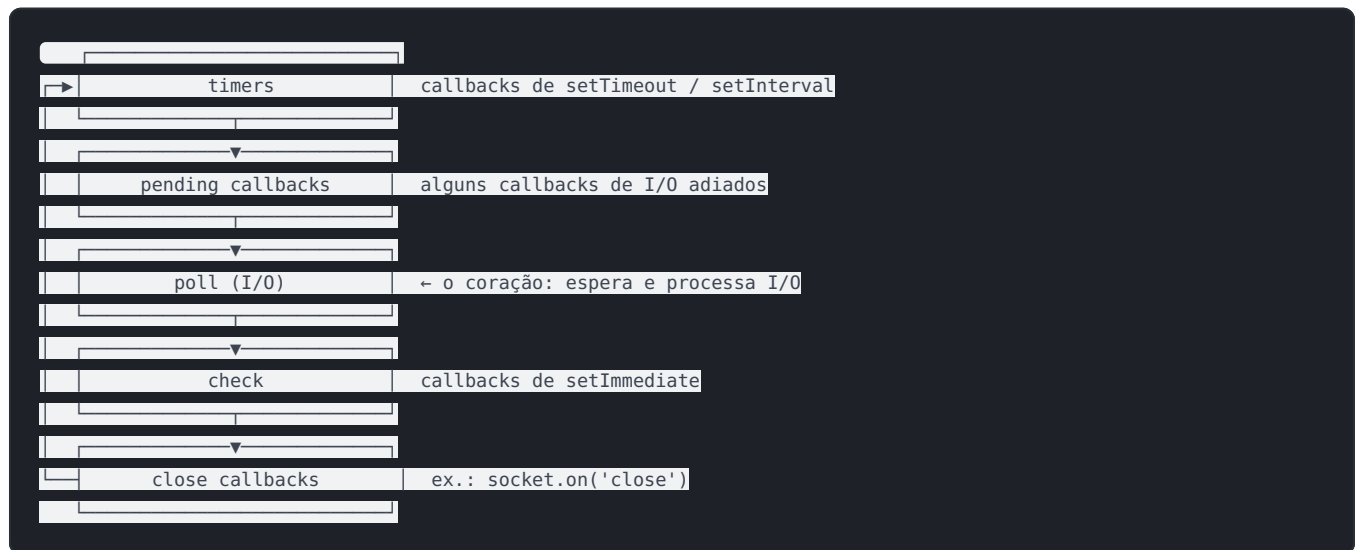
A saída é **A E D C B**. Se você previu diferente, é aqui que mora quase toda confusão sobre o Node.

Primeiro, todo o **código síncrono** roda até o fim: **A**, depois **E**. Nada assíncrono compete com isso.

Aí vem a parte não óbvia. Antes de o loop tocar qualquer "fase", ele esvazia duas filas prioritárias, as **microtasks**: primeiro a de `process.nextTick` (**D**), depois a de Promises (**C**). Só então o loop entra nas suas fases e roda o timer (**B**).

O modelo mental que resolve 90% dos casos: **síncrono** → **nextTick** → **promises** → **resto**.

Aprofundando — as fases do loop. O "resto" tem estrutura. O event loop do libuv processa callbacks em fases de ordem fixa, ciclando enquanto houver trabalho pendente:



O detalhe que separa quem decorou de quem entendeu: as microtasks drenam **entre cada fase**, não só no fim do ciclo. Por isso um `nextTick` agendado dentro de um callback de timer roda antes de o loop sair da fase de timers.

3. Por que `setImmediate` e `setTimeout(fn, 0)` às vezes trocam de ordem?

Antes de ler: você esperaria que "timeout zero" fosse sempre o mais rápido. Não é. Por quê?

O loop tem ordem fixa de fases. Duas importam aqui: **timers** (onde `setTimeout` roda) e **check** (onde `setImmediate` roda). A fase check vem logo depois da fase de I/O (poll).

Então a pergunta que decide tudo é: *de onde* o código foi agendado?

- **Do nível superior do script:** é uma corrida. Depende de quanto tempo o processo levou para chegar na fase de timers desde o início. A ordem entre os dois **não é garantida**.
- **De dentro de um callback de I/O:** aí `setImmediate` **sempre** ganha. Porque o callback de I/O roda na fase poll, e a próxima fase é check — o immediate roda antes de o loop dar a volta inteira e voltar aos timers.

```
import readFile from 'node:fs'
|
```

```

readFile 'arquivo.txt'
  setTimeout console log 'timeout' 0
  setImmediate console log 'immediate'
}
// Dentro do callback de I/O, a saída é SEMPRE:
// immediate
// timeout

```

Se você consegue explicar essa ordem, provou que entendeu que o loop tem fases e que a posição no ciclo é o que decide — não a "prioridade" dos temporizadores.

4. Se é uma thread só, quem roda `fs.readFile` e `crypto` ?

Pense na contradição: rede o Node faz com `epoll/kqueue`, sem gastar thread. Mas ler um arquivo? Não existe forma universalmente assíncrona de fazer isso no sistema operacional. Então como o Node não bloqueia ao ler disco?

A resposta desmonta a ideia de "single-threaded": o libuv mantém um **thread pool** escondido, com **4 threads** por padrão (`UV_THREADPOOL_SIZE`). Operações que o SO não oferece assíncronas rodam ali, fora da thread principal, e só o callback volta ao event loop quando terminam. As três famílias que usam o pool:

- **Arquivo** — `fs.readFile`, `fs.writeFile`, etc.
- **DNS** — `dns.lookup` (atenção: `dns.resolve` usa a rede, não o pool).
- **Crypto pesada** — `pbkdf2`, `scrypt`, `randomBytes`.

Aprofundando — o gargalo invisível. Como o pool tem 4 threads, dispare **5** operações de `crypto.pbkdf2` ao mesmo tempo e a quinta espera na fila até uma das quatro liberar:

```

import crypto from 'node:crypto'
|
// 5 operações, pool de 4 → a quinta serializa
for let i = 0 i < 5 i++
  console time `pbkdf2 ${i}`
  crypto.pbkdf2('senha', 'sal', 1000000, 64, 'sha512')
  console timeEnd `pbkdf2 ${i}` // a 5ª leva ~2x o tempo das outras
|
|

```

Isso é uma pergunta clássica: "o Node é single-threaded?" A resposta completa é *não no I/O* — há 4 threads do libuv fazendo arquivo, DNS e crypto por baixo.

5. Qual é o pior jeito de quebrar um servidor Node?

Pense: já que a thread larga tarefas que *esperam*, o que acontece com uma tarefa que **não espera** — que só calcula, sem parar?

Ela trava tudo. Um loop síncrono pesado — consolidar 12 mil pagamentos, comprimir um payload gigante, um `JSON.stringify` de um objeto enorme — segura a thread do começo ao fim. Durante esse tempo, o event loop **não avança**: nenhuma outra requisição é atendida, o servidor inteiro congela. Este é o modo de

falha assinatura do Node — latência que dispara não por carga de I/O, mas por um handler que trava a thread.

Ligado à sua âncora. Considere o endpoint do ERP que gera um relatório financeiro consolidando milhares de pagamentos num loop síncrono:

```
// PROBLEMA: bloqueia a thread durante todo o cálculo.
// Enquanto roda, até um /login concorrente fica preso na fila.
app.get('/relatorio', req, res => {
  const dados = consolidarPagamentos(todosOsPagamentos) // síncrono, pesado
  res.json(dados)
})

// MELHOR: move o trabalho pesado para um worker thread.
import { Worker } from 'node:worker_threads'

app.get('/relatorio', req, res => {
  const worker = new Worker('./consolidacao-worker.js', {
    workerData: { pagamentos: todosOsPagamentos }
  })

  worker.on('message', (dados) => {
    res.json(dados)
  })
  worker.on('error', (err) => {
    res.status(500).json({ erro: err.message })
  })
})
```

O passo que separa pleno de júnior vem antes do código: como você *soube* que o event loop estava bloqueado? A resposta não é "achei que estava lento" — é medir o **event loop lag** (via `perf_hooks.monitorEventLoopDelay`) e ver o atraso disparar sob concorrência, isolando que o gargalo era CPU na thread principal, não espera de I/O.

Alternativas ao worker thread, com seus trade-offs: - **Quebrar o trabalho em pedaços** e ceder o controle entre eles (`setImmediate` como ponto de yield). Simples, mas frágil e ainda compete com o resto na mesma thread. - **Delegar para outro serviço/processo**. Isola de vez, ao custo de complexidade operacional (fila, comunicação, deploy). - **Worker thread**. O meio-termo certo para trabalho pesado recorrente dentro do mesmo processo.

Teste-se antes do portão

Se você responde estas de cabeça, com o "porquê", o tópico fechou:

1. Por que uma thread só escala para milhares de conexões — e em que tipo de carga isso **deixa** de valer?
2. Ordene e justifique: síncrono, `nextTick`, `Promise`, `setTimeout(fn, 0)`, `setImmediate`.
3. Quando `setImmediate` vence `setTimeout(fn, 0)` de forma **garantida**, e por quê?
4. O Node é single-threaded? Defenda a resposta completa, incluindo o thread pool.
5. Descreva, do sintoma à verificação, como você diagnosticaria um event loop bloqueado em produção — com o passo de medição no meio.

Onde o entrevistador vai cutucar

- "**`process.nextTick` pode ser perigoso?**" — Sim: recursão de `nextTick` faminta (starve) o event loop, impedindo que ele avance para as fases de I/O.
- "**Aumentar `UV_THREADPOOL_SIZE` sempre ajuda?**" — Não: acima do número de cores úteis, você troca um gargalo por contenção de CPU.
- "**`dns.lookup` e `dns.resolve` são a mesma coisa?**" — Não: `lookup` usa o thread pool; `resolve` usa a rede diretamente. Confundir os dois esconde um gargalo.